

Adicionar campo Rendimento (Porções) ao Recipe

BlazorAppFood · 27 de Maio de 2026

Descrição da Task

A task SCRUM-35 pedia a adição de um campo **Servings** (Rendimento / Porções) à tabela *Recipe* da base de dados, com propagação completa pela aplicação: migration de base de dados, modelo C#, repositório Dapper, formulários de criação e edição, e página de detalhe da receita.

Resumo das Alterações

Ficheiro	Tipo de alteração
Models/Recipe.cs	Adicionada propriedade Servings
Migrations/20260527000000_AddServingsToRecipe.cs	Nova migration (ficheiro criado)
Repositories/RecipeRepository.cs	CreateRecipe e UpdateRecipe atualizados
Pages/AddRecipe.razor	Campo 'Rendimento (Porcoes)' adicionado ao form
Pages/EditRecipe.razor	Campo 'Rendimento (Porcoes)' adicionado ao form
Pages/RecipePage.razor	Exibe porcoes quando Servings > 0

Detalhe das Alterações

1. Models/Recipe.cs

Adicionada nova propriedade **Servings** do tipo *int* ao modelo Recipe. Esta propriedade representa o número de porções que a receita rende.

Models/Recipe.cs – linha adicionada

```
public int Servings { get; set; }
```

2. Migrations/20260527000000_AddServingsToRecipe.cs

Criado novo ficheiro de migration EF Core. O método *Up* executa um **ALTER TABLE** para adicionar a coluna *Servings* **INT NOT NULL DEFAULT 0** na tabela *Recipe*. O método *Down* remove a coluna, permitindo reverter a migration.

Migrations/20260527000000_AddServingsToRecipe.cs – ficheiro novo

```
migrationBuilder.Sql("ALTER TABLE Recipe ADD Servings INT NOT NULL DEFAULT 0");
```

Para aplicar a migration, executar na pasta do projeto:

```
dotnet ef database update
```

3. Repositories/RecipeRepository.cs

O repositório usa *Dapper* com stored procedures e SQL inline. Como o stored procedure *spRecipe_Insert* não foi alterado, o campo *Servings* é persistido através de um **UPDATE** inline executado dentro da mesma transacao após o SP retornar o ID da nova receita.

CreateRecipe — **UPDATE** após *spRecipe_Insert*:

RecipeRepository.cs – dentro da transacao de CreateRecipe

```
await conn.ExecuteAsync( "UPDATE Recipe SET Servings = @Servings WHERE Id_Recipe = @Id_Recipe", new { Servings = recipe.Servings, Id_Recipe = recp_id }, transaction, commandType: CommandType.Text);
```

UpdateRecipe — *Servings* incluído no **SET**:

RecipeRepository.cs – UpdateRecipe

```
UPDATE Recipe SET NameRecipe = @NameRecipe, Prep_Time = @Prep_Time, Cook_Time = @Cook_Time, Preparation = @Preparation, Servings = @Servings, [Image] = COALESCE(@Image, [Image]) WHERE Id_Recipe = @Id_Recipe
```

4. Pages/AddRecipe.razor

Adicionado campo numérico **Rendimento (Porções)** no formulário de criação de receita, posicionado antes dos campos *Prep Time* e *Cook Time*. O campo usa *@bind* para ligar diretamente à propriedade *recipe.Servings*.

AddRecipe.razor – HTML adicionado

```
<div class="mb-3"> <label class="form-label">Rendimento (Porcoes)</label> <input type="number" min="1" max="999" @bind="recipe.Servings" class="form-control" required /> </div>
```

5. Pages/EditRecipe.razor

Adicionado o mesmo campo **Rendimento (Porções)** no formulário de edição. Ao carregar a receita existente, o valor de *Servings* já vem preenchido automaticamente através do *recipe = await RecipeRepository.Recipe_GetOne(id)*.

EditRecipe.razor – HTML adicionado

```
<div class="mb-3"> <label class="form-label">Rendimento (Porcoes)</label> <input  
type="number" min="1" max="999" @bind="recipe.Servings" class="form-control"  
required /> </div>
```

6. Pages/RecipePage.razor

Na página de detalhe da receita, o rendimento é mostrado condicionalmente: apenas aparece se *recipe.Servings > 0*, evitando exibir '0 porções' em receitas antigas que ainda não têm o campo preenchido.

RecipePage.razor – bloco adicionado

```
@if (recipe.Servings > 0) { <p><b>Rendimento:</b> @recipe.Servings porcoes</p> }
```

Notas Importantes

As queries que usam **SELECT *** (como *GetRecipeInfo*, *RecipesList*, e *GetFavoriteRecipes*) irão buscar automaticamente a coluna *Servings* após a migration ser aplicada, sem necessidade de alteração adicional.

O stored procedure *spRecipe_Insert* não foi modificado — a estratégia de UPDATE inline na mesma transacao garante consistência sem necessidade de acesso ao servidor para alterar o SP.